

# CSCI 112

## Programming with C

### LECTURE: 6

J. L. Pach

# OUTLINE:

- Function
- Variables
- Variable scope

# FUNCTIONS

# PROTOTYPE & ITS FUNCTION

- In contrast to function implementations, function prototypes are terminated with a semicolon( `;` ). The function name is `symbolic name` .
- A minimal prototype must precede the function definition, and it is standard to list function prototypes alphabetically after preprocessor directives within a file
- The return type can be any of the data types presented in the previous material, and additionally, void can be used if no value is to be returned.

```
return-type function-name(only type of parameter declarations, if any);

return-type function-name(parameter declarations, if any)
{
        statements;                     /* including declarations */
        return expression1;
}
```

# MAIN() EXAMPLE

- The `main()` function is the entry point of a C program. When a C program is executed, the main function is called first.
- The main function is an excellent example. The type of value it returns is `int`, and its `symbolic name` is `main`. In this variant, the function doesn't accept any parameters, though another version with `argc` does exist. The final line, `return 0;`, instructs the compiler that the function is returning a value of `0`.

```c
#include <stdio.h>

int main()
{
  /* body*/
  return 0;
}
```

# EXIT(0)

One might wonder why it returns zero if we never use this value anywhere. In fact, it isn't truly unused. When the program finishes its execution, control is handed back to the computer's operating system, which can respond to the returned value. A `return` value of `0` signifies that the program completed successfully, as the programmer intended, without errors. Any other value indicates an abnormal termination or an error.

```c
#include <stdio.h>

int main()
{
        /* body*/
  return 0;
}
```



# FUNCTIONS, RETURN & CASTING 1/3

Regardless of the function's return type, the compiler expects the return keyword followed by a value of the same type as specified in the function's declaration or prototype. Therefore, if a function returns `void` (nothing), the last line should simply be `return;`.

```c
#include <stdio.h>

int main()
{
  /* body*/
}
```



# FUNCTIONS, RETURN & CASTING 2/3

Similar to **casting**, there is **explicit casting** (consciously performed by the programmer) and **implicit casting** (done automatically by the compiler). If the result of an int expression is assigned to a float variable, implicit casting will occur. Another programmer analyzing this code won't know if this was a mistake or intentional. To avoid this ambiguity, it's a best practice to use explicit casting with parentheses, for example:

```c
float x = (float) (y + z);
```



```c
#include <stdio.h>

int main()
{
        /* body*/
}
```



# FUNCTIONS, RETURN, AND CASTING 3/3

- The return statement works in a similar way. If a programmer omits the return instruction, the compiler will return "some" value (undefined behavior) depending on its settings (usually the value from the processor's ax register). This can lead to undesirable consequences and hard-to-find bugs, since the compiler won't flag them.

  > For this reason, this course requires every function to end with the return keyword and a variable of the function's declared type.

- Finally, it's worth remembering that the terms **procedure**, **method**, and **function** are often used interchangeably. The main difference is that a function, like the main function, is not nested within another structure. In other languages, you can nest functions within other entities, and these are then called methods. A procedure, on the other hand, is the ancestor of a function in assembly language.

# PROTOTYPE & ITS FUNCTION - MYFUNCTION1 1/2

```c
#include <stdio.h>
void myFunction1(void);
void myFunction1()
{
  printf("Text from myFuntion1\n");
  return;
}
int main()
{
  myFunction1();
  return 0;  /* <---- Correct exit */
}
```



```
Text from myFuntion1
```

# PROTOTYPE & ITS FUNCTION - MYFUNCTION1 2/2

- A function prototype must appear **before** its definition.
- A prototype ends with a semicolon.
- If the function does not take any parameters, you can write void in the prototype, but nowadays few people do this, because the compiler does not require it.

```c
#include <stdio.h>

int myFunction2(int);

int myFunction2(int n)
{
  n++;
  printf("incremented n from myFuntion2 equals %d\n", n);
  return 1;
}
int main()
{
  int result = myFunction2(2);
  printf("result of myFuntion2 equals %d\n", result);
  return 0;
}
```

```
incremented n from myFuntion2 equals 3
result of myFuntion2 equals 1
```

# PROTOTYPE & ITS FUNCTION – MYFUNCTION2 2/2

- `myFunction2` is of type `int`, so the value it returns is also of type `int`. Remember that everything inside the parentheses of the function parameters is a **copy** of the value passed to the function. This means that `myFunction2` literally declares `int n` before the first statement of the function and assigns it the value 2 (like `int n = 2;`).
- As known from the concept of a block, once the body of `myFunction2` finishes executing, this local variable n is destroyed — it exists only within the scope of the function. Inside the function, the value of n is incremented by 1, displayed using `printf`, and the function then returns the value 1. This returned value is assigned to result in main.
- This demonstrates how a function communicates with a variable in `main`: the parameter n is a local copy, and the function can return a value to the caller.



```c
#include <stdio.h>
int  myFunction3(void);

int myFunction3()
{
  int n = 2;
  printf("n from myFuntion3 equals %d\n", n);
  /* the compiler will not return an error if     */
  /* we omit return, and the function will return */
  /* an unspecified value                         */
}
int main()
{
  int result = myFunction3();
  printf("result of myFuntion3 equals %d\n", result);
  return 0; /* <--- Correct exit */
}
```

```
n from myFuntion3 equals 2
result of myFuntion3 equals 27
```

```c
int main()
{
  myFunction1();
  int result = myFunction2(2);
  printf("result of myFuntion2 equals %d\n", result);
  result = myFunction3();
  printf("result of myFuntion3 equals %d\n", result);
  return 0; /* <--- Correct exit */
}
```

```
Text from myFuntion1
incremented n from myFuntion2 equals 3
result of myFuntion2 equals 1
n from myFuntion3 equals 2
result of myFuntion3 equals 27
```

```c
#include <stdio.h>

void myFunction1(void);
int  myFunction2(int);
int  myFunction3(void);

void myFunction1()
{
  printf("Text from myFuntion1\n");
  return;
}
int myFunction2(int n)
{
  n++;
  printf("incremented n from myFuntion2 equals %d\n", n);
  return 1;
}
int myFunction3()
{
  int n = 2;
  printf("n from myFuntion3 equals %d\n", n);
  /* the compiler will not return an error if     */
  /* we omit return, and the function will return */
  /* an unspecified value                         */
}
```

# VARIABLES

# VARIABLES

- **variable scope** – the region in which a variable is valid; any variable must be declared before its **first** use;
- **Variables**:
  - local (**automatic**)
  - global
- **local variable** – is only accessible within the block where it is defined. Once the block ends, access to the local variable is lost.
- **global variable** – declared outside of functions, is accessible from any point in the program below its declaration.



```c
#include <stdio.h>
int x = 5;              /* global x */

void myFunction1(void);
void myFunction1()
{
  printf("x equals %d that is read by function myFunction\n", x);
}
int main()
{
  int * pointerGlobalX = &x;
  myFunction1();
  int x = 3;            /* local x */
  printf("x equals %d that is read by function main\n", x);
  printf("x equals %d that is read by function main\n", *pointerGlobalX);
  return 0;
} /* We don't have direct access to a shadowed variable. The only way to access it is through a pointer. */
```

```
x equals 5 that is read by function myFunction
x equals 3 that is read by function main
x equals 5 that is read by function main
```

# SUMMARY - VARIABLE SCOPE

- Global variables are accessible throughout a program, but they can be temporarily hidden by a local variable declared within a nested block, such as a function, for loop, while loop, or even an if statement.
- This is known as "variable shadowing" and within the local variable's scope, any reference to that variable name will refer to the local one. The global variable remains inaccessible until the local variable's block is exited and the local variable is released from memory. At that point, access to the global variable is restored.

# THANK

You